INFORME DE LABORATORIO # 3

Gramáticas Libres de Contexto

Antecedente	Información
Asignatura	Teoría de la Computación – INFO1148
Carrera	Ingeniería Civil en Informática
Docente	Marcos Lévano
Integrantes	Juan Cárdenas y Bayron Flores (Jefe de Grupo)
Fecha	21 de octubre de 2026

1. Introducción

El objetivo del laboratorio es diseñar, analizar y validar una Gramática Libre de Contexto (GLC) que represente la estructura sintáctica de un lenguaje de programación simplificado. Se identifican los terminales, los no terminales y el símbolo inicial; se define formalmente G = (V, Σ, P, S); se construyen las producciones para programas con una o más instrucciones (asignaciones simples, asignaciones con una operación aritmética y salidas); y se comprueba la coherencia de la gramática mediante derivaciones por la izquierda, un árbol de derivación y la clasificación de cadenas válidas e inválidas. Los conceptos centrales son símbolo inicial, lenguaje generado L(G), derivación, árbol sintáctico y la separación entre análisis léxico (identificadores y números) y análisis sintáctico (orden de las categorías). Como extensión creativa se incorporan declaraciones de variables de tipo entero y real.

2. Desarrollo

2.1 Pregunta 1 – Componentes de la gramática

a) Símbolos terminales

Categoría	Elementos identificados
Palabras reservadas	`Begin, Programa, End, salida`
Operadores	`+ - * / %` (aritméticos) y `=` (asignación)
Símbolos especiales	# (terminador de instrucción)
Identificadores	`ID`: categoría léxica de nombres como x, z2 y operativo_01_02. Patrón: L (L | D | _)*, con L = letra y D = dígito.
Números	`NUM`: categoría léxica de valores como 20, 0 y 1.345. Patrón: D+ (. D+)?, sin signo.

b) Símbolos no terminales

```
V = { <programa>, <lista_instrucciones>, <instruccion>, <asignacion_simple>,
<asignacion_aritmetica>, <expresion>, <operando>, <operador>, <salida> }
```

Se usan dos no terminales para las asignaciones (<asignacion_simple> y <asignacion_aritmetica>) en lugar de uno solo, porque el enunciado pide diferenciarlas explícitamente (punto 7); ambos cumplen el rol de <asignacion> sugerido.

c) Símbolo inicial

S = <programa>

d) Definición formal de la gramática

G = (V, Σ, P, S)

V = { <programa>, <lista_instrucciones>, <instruccion>, <asignacion_simple>, <asignacion_aritmetica>, <expresion>, <operando>, <operador>, <salida> } (9 no terminales)

Σ = { Begin, Programa, End, salida, ID, NUM, =, #, +, -, *, /, % } (13 terminales)

P = Ver producciones desarrolladas en la Pregunta 2 (9 reglas que, al separar sus alternativas, suman 17 producciones).

S = <programa>

e) Análisis

No se enumeran los identificadores ni los números dentro de la gramática porque son conjuntos infinitos o no acotados: existen nombres nuevos (x, z2, temporal, operativo_01_02…) y números de longitud arbitraria (20, 0, 1.345, 1000.25…). Listar solo algunos valores (por ejemplo ID → x | y | z) haría que la gramática rechazara programas correctos, y listarlos todos es imposible. Por eso se modelan como categorías léxicas con un patrón general (ID y NUM), y cada token conserva su lexema como dato asociado: en la notación ID[z], el terminal es ID y z es su lexema. No se añaden reglas como ID → x ni NUM → 20.

Es una GLC porque el lado izquierdo de cada producción es un único no terminal. Decisiones léxicas: las palabras reservadas distinguen mayúsculas y minúsculas (un nombre como salida_total sigue siendo ID); NUM admite enteros y reales sin signo (el 0 aparece en el programa de referencia, así que se interpreta «positivos» como «no negativos»); los espacios y los comentarios /* … */ los descarta el analizador léxico y no forman parte de la GLC. Los patrones de ID y NUM se expresan además como producciones a nivel de carácter en el Anexo A.1.

2.2 Pregunta 2 – Construcción de la Gramática Libre de Contexto

a) Producciones propuestas

No terminal	Producción
<programa>	<programa> → Begin Programa ID <lista_instrucciones> End
<lista_instrucciones>	<lista_instrucciones> → <instruccion> <lista_instrucciones> | <instruccion>
<instruccion>	<instruccion> → <asignacion_simple> | <asignacion_aritmetica> | <salida>
<asignacion_simple>	<asignacion_simple> → ID = <operando> #
<asignacion_aritmetica>	<asignacion_aritmetica> → ID = <expresion> #
<expresion>	<expresion> → <operando> <operador> <operando>
<operando>	<operando> → ID | NUM
<operador>	<operador> → + | - | * | / | %
<salida>	<salida> → salida ID #

Diferenciación explícita de las tres instrucciones

- **Asignación simple:** ID = <operando> #. Copia un identificador o un número a un destino, sin operador (x = 20 #, temporal = x #).
- **Asignación con expresión aritmética:** ID = <operando> <operador> <operando> #. Contiene exactamente un operador entre dos operandos (z = x + y #).
- **Instrucción de salida:** salida ID #. Empieza con la palabra reservada salida y no lleva el operador = (salida x #).

La gramática no es ambigua: tras ID = <operando>, el siguiente terminal decide la regla (# cierra una asignación simple; un operador inicia una asignación aritmética) y salida solo puede iniciar la tercera clase. La recursión de <lista_instrucciones> exige al menos una instrucción y permite cualquier cantidad de ellas.

b) Explicación de los no terminales

No terminal	Función dentro de la gramática
`<programa>`	Símbolo inicial. Construye un programa completo: cabecera Begin Programa ID, cuerpo y cierre End.
`<lista_instrucciones>`	Produce una o más instrucciones en orden (recursión por la derecha); nunca deriva ε.
`<instruccion>`	Selecciona el tipo de instrucción: asignación simple, asignación aritmética o salida.
`<asignacion_simple>`	Asigna un operando (ID o NUM) a un identificador destino y termina con #.
`<asignacion_aritmetica>`	Asigna al destino el resultado sintáctico de una expresión de una operación binaria.
`<expresion>`	Une un operando izquierdo, un operador y un operando derecho.
`<operando>`	Selecciona un identificador (ID) o un número (NUM).
`<operador>`	Selecciona uno de los cinco operadores aritméticos: + - * / %.
`<salida>`	Forma la instrucción salida ID #, que imprime el valor de un identificador.

c) Verificación de la gramática

Sí. El programa de referencia tiene la cabecera Begin Programa operativo_01_02, veinte instrucciones y el cierre End. Tras descartar los espacios y el comentario /* operaciones y salidas */ (que elimina el analizador léxico; la GLC no lo genera), cada instrucción coincide con una producción: 6 asignaciones simples (x = 1.345 #, y = 20 #, temporal = 0 #, temporal = x #, x = y #, y = temporal #), 7 asignaciones aritméticas (z y z2 con +; h y h2 con -; k con *; t con /; m con %) y 7 salidas (z, h, k, t, m, x, y). La lista se deriva con 20 aplicaciones de <lista_instrucciones>. El validador acepta el archivo completo ejemplos/programa_referencia.txt (91 tokens y 20 nodos <instruccion> en el árbol). Alcance: cada asignación aritmética tiene una sola operación binaria; no hay paréntesis, operaciones encadenadas ni signo unario (x = a + b * c # y x = -1 # no pertenecen a L(<instruccion>)).

Pertenencia a L(<programa>) y a L(<instruccion>). Como S = <programa>, el lenguaje generado es L(G) = L(<programa>): solo programas completos (Begin Programa … End). Una instrucción suelta, como z = x + y #, pertenece a L(<instruccion>) pero no a L(<programa>). Para una instrucción w se cumple que w ∈ L(<instruccion>) si y solo si Begin Programa prueba w End ∈ L(<programa>). En las Preguntas 3 y 4 cada cadena se comprueba contra el lenguaje que se indica, y las derivaciones y los árboles se muestran tanto desde <instruccion> como desde el símbolo inicial <programa>.

2.3 Pregunta 3 – Derivaciones y árbol sintáctico

En cada paso se sustituye el no terminal situado más a la izquierda. De cada cadena se muestran dos derivaciones: desde <instruccion>, que prueba que pertenece a L(<instruccion>), y desde <programa> (símbolo inicial S), con la instrucción dentro del programa mínimo Begin Programa prueba … End. Las formas sentenciales conservan el terminal de categoría ID; el último paso es la sustitución léxica por los lexemas concretos (Anexo A.1), que elige el valor del token y no es una producción de P.

a) Derivación por la izquierda para:

```
z = x + y #
```

Desde `<instruccion>` (la cadena pertenece a L(<instruccion>)):

```
<instruccion>
⇒ <asignacion_aritmetica>
⇒ ID = <expresion> #
⇒ ID = <operando> <operador> <operando> #
⇒ ID = ID <operador> <operando> #
⇒ ID = ID + <operando> #
⇒ ID = ID + ID #
⇒ z = x + y #
```

Reglas aplicadas, en orden: <instruccion> → <asignacion_aritmetica>; <asignacion_aritmetica> → ID = <expresion> #; <expresion> → <operando> <operador> <operando>; <operando> → ID; <operador> → +; <operando> → ID.

Desde `<programa>` (símbolo inicial S), dentro de Begin Programa prueba … End:

```
<programa>
⇒ Begin Programa ID <lista_instrucciones> End
⇒ Begin Programa ID <instruccion> End
⇒ Begin Programa ID <asignacion_aritmetica> End
⇒ Begin Programa ID ID = <expresion> # End
⇒ Begin Programa ID ID = <operando> <operador> <operando> # End
⇒ Begin Programa ID ID = ID <operador> <operando> # End
⇒ Begin Programa ID ID = ID + <operando> # End
⇒ Begin Programa ID ID = ID + ID # End
⇒ Begin Programa prueba z = x + y # End
```

Reglas aplicadas: <programa> → Begin Programa ID <lista_instrucciones> End; <lista_instrucciones> → <instruccion>; <instruccion> → <asignacion_aritmetica>; <asignacion_aritmetica> → ID = <expresion> #; <expresion> → <operando> <operador> <operando>; <operando> → ID; <operador> → +; <operando> → ID.

b) Derivación para:

```
salida z #
```

Desde `<instruccion>` (la cadena pertenece a L(<instruccion>)):

```
<instruccion>
⇒ <salida>
⇒ salida ID #
⇒ salida z #
```

Reglas aplicadas, en orden: <instruccion> → <salida>; <salida> → salida ID #.

Desde `<programa>` (símbolo inicial S), dentro de Begin Programa prueba … End:

```
<programa>
⇒ Begin Programa ID <lista_instrucciones> End
⇒ Begin Programa ID <instruccion> End
⇒ Begin Programa ID <salida> End
⇒ Begin Programa ID salida ID # End
⇒ Begin Programa prueba salida z # End
```

Reglas aplicadas: <programa> → Begin Programa ID <lista_instrucciones> End; <lista_instrucciones> → <instruccion>; <instruccion> → <salida>; <salida> → salida ID #.

c) Árbol de derivación

Árbol de derivación de z = x + y # con raíz <instruccion> (Figura 1) y el mismo árbol dentro del programa mínimo, con raíz el símbolo inicial <programa> (Figura 2).

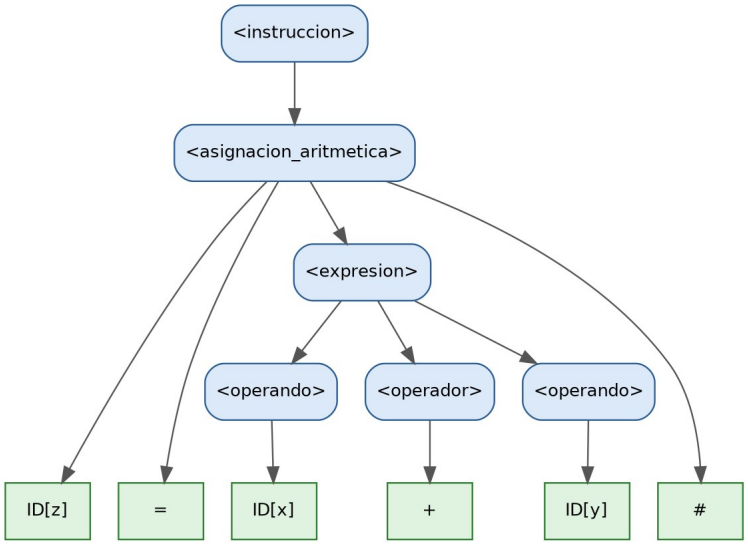


Figura 1. Árbol de derivación de z = x + y # desde <instruccion>. No terminales en azul (nodos internos); terminales en verde (hojas). Generado con Graphviz a partir de la estructura que devuelve el validador. No se utilizó JFLAP.

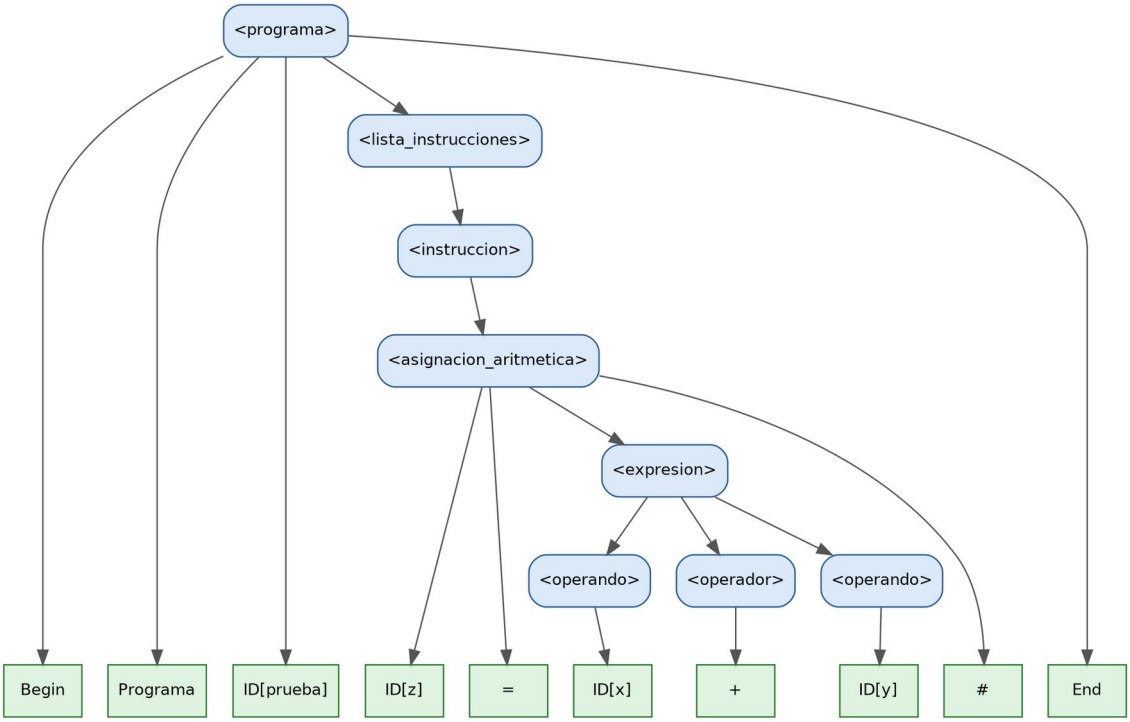


Figura 2. Árbol de derivación desde <programa> (raíz = símbolo inicial S), con la instrucción dentro de Begin Programa prueba … End.

d) Explicación del árbol

En el árbol de la Figura 1 la raíz es `<instruccion>` y en el de la Figura 2 es `<programa>`, el símbolo inicial S, que representa el programa completo. Los nodos internos son siempre símbolos no terminales: cada uno se expande en sus hijos aplicando exactamente una producción, y el conjunto de hijos, leído de izquierda a derecha, es el lado derecho de esa producción (por ejemplo, `<expresion>` tiene como hijos `<operando> <operador> <operando>` según `<expresion> → <operando> <operador> <operando>`).

Las hojas son los símbolos terminales y no tienen hijos. En la Figura 1, leídas de izquierda a derecha, forman la cadena derivada: `ID[z], =, ID[x], +, ID[y],` #, es decir, z = x + y #. En la Figura 2 las hojas son

`Begin, Programa, ID[prueba], ID[z], =, ID[x], +, ID[y], #, End`, y forman el programa Begin Programa prueba z = x + y # End; la subcadena z = x + y # corresponde al subárbol de `<instruccion>`. Cada árbol coincide paso a paso con la derivación por la izquierda correspondiente de la sección a): cada expansión de un no terminal es un nodo interno con sus hijos. Los corchetes ID[z] indican el lexema del token y no son símbolos adicionales.

e) Validación de cadenas

Cadena	¿Puede generarse?	Justificación
`x = 20 #`	Sí, ∈ L(<instruccion>)	<instruccion> ⇒ <asignacion_simple> ⇒ ID = <operando> # ⇒ ID = NUM # ⇒ x = 20 #.
`z = x * y #`	Sí, ∈ L(<instruccion>)	<instruccion> ⇒ <asignacion_aritmetica> ⇒ ID = <expresion> # ⇒ ID = <operando> <operador> <operando> # ⇒ ID = ID <operador> <operando> # ⇒ ID = ID * <operando> # ⇒ ID = ID * ID # ⇒ z = x * y #. El operador * es una alternativa de <operador>.
`salida temporal #`	Sí, ∈ L(<instruccion>)	<instruccion> ⇒ <salida> ⇒ salida ID # ⇒ salida temporal #. temporal es un ID válido.

Las tres cadenas se comprueban contra L(<instruccion>). Por sí solas no pertenecen a L(<programa>), porque les faltan Begin Programa … End; dentro del programa mínimo Begin Programa prueba … End sí forman programas válidos de L(G).

2.4 Pregunta 4 – Validación y detección de errores

Los casos 1 a 7 son instrucciones y se comprueban contra L(<instruccion>); el caso 8 es un programa completo y se comprueba contra L(<programa>). Ninguna de las instrucciones 1 a 3 pertenece a L(<programa>) por sí sola. Las posiciones de error (línea, columna) las informa validador.py (--fragmento para los casos 1 a 7).

Caso	¿Pertenece al lenguaje?	Justificación / error detectado
Caso 1 `x = 10 #`	Sí, ∈ L(<instruccion>)	Asignación simple: ID = NUM #. Todos los símbolos están en su lugar.
Caso 2 `z = x + y #`	Sí, ∈ L(<instruccion>)	Asignación aritmética: ID = ID + ID #, con un operador entre dos operandos.
Caso 3 `salida z #`	Sí, ∈ L(<instruccion>)	Salida: salida ID #.
Caso 4 `x 10 #`	No, ∉ L(<instruccion>)	Falta el operador de asignación = entre x y 10; tras el identificador se encontró el número 10 (línea 1, columna 3).
Caso 5 `z = + x y #`	No, ∉ L(<instruccion>)	Operador mal ubicado: después de = se requiere un operando (ID o NUM) y aparece + (línea 1, columna 5). La expresión es infija: <operando> <operador> <operando>.
Caso 6 `salida #`	No, ∉ L(<instruccion>)	Falta el identificador que debe seguir a salida; se encontró # (línea 1, columna 8).
Caso 7 `x = 20`	No, ∉ L(<instruccion>)	Falta el terminador #: la entrada termina donde se esperaba # o un operador (línea 1, columna 7).
Caso 8 `Begin / Programa prueba / x = 5 # / salida x # / End`	Sí, ∈ L(<programa>)	Programa completo: Begin Programa ID <lista_instrucciones> End, con una asignación simple y una salida. Las barras del enunciado se leen como separadores de línea (ver nota bajo la tabla).

Nota sobre el caso 8. El enunciado escribe el programa con barras / entre sus componentes. Se leen como separadores de línea, que es la única lectura bajo la cual la cadena es un programa; el programa queda así:

```
Begin
Programa prueba
x = 5 #
salida x #
End
```

Si las barras fueran caracteres literales, el validador rechazaría la cadena en línea 1, columna 7, porque aparece / donde la producción exige Programa. La barra no es separador del lenguaje: dentro de una expresión es división y /* inicia un comentario. Ambas lecturas constan en el Anexo A.2.

Análisis general:

Las cadenas incorrectas (casos 4 a 7) fallan por un único elemento sintáctico faltante o mal ubicado, y el análisis las detecta en el primer símbolo que ninguna producción puede continuar. Caso 4: falta el terminal =. Caso 5: el operador + está antes del primer operando (posición incorrecta; además x e y van después del operador, en notación prefija, y la gramática exige notación infija). Caso 6: falta el ID después de salida. Caso 7: falta el terminador #. Los casos válidos son: 1, 2 y 3, que pertenecen a L(<instruccion>) como asignación simple, asignación aritmética y salida, y el caso 8, que pertenece a L(<programa>) como programa completo. Todas las decisiones son sintácticas: no se evalúan valores ni se comprueba si una variable fue definida.

3. Extensión de la gramática

Construcción seleccionada: **Opción C – Declaración de variables**

Nueva sintaxis propuesta:

`entero x #` y `real temperatura #`: una declaración es un tipo (entero o real), un identificador y el terminador #. Es una instrucción más, que puede aparecer en cualquier posición de la lista de instrucciones.

Modificaciones realizadas a la gramática:

Elemento	Modificación
Nuevos terminales	`entero, real` (palabras reservadas, solo cuando la extensión está activa).
Nuevos no terminales	`<declaracion>, <tipo>`

Nuevas producciones	`<declaracion> → <tipo> ID #`
	`<tipo> → entero | real`
	`Modificada: <instruccion> → <asignacion_simple> |`
	`<asignacion_aritmetica> | <salida> | <declaracion>`

Ejemplo de cadena válida:

`entero edad #` (también real temperatura #). Es válida porque tiene tipo, identificador y terminador.

Ejemplo de cadena inválida:

`entero #`: inválida; falta el identificador después del tipo (línea 1, columna 8).

Derivación de ejemplo:

```
<instruccion>
⇒ <declaracion>
⇒ <tipo> ID #
⇒ entero ID #
⇒ entero edad #
```

Resultado: entero edad # ∈ L(<instruccion>) y, dentro de Begin Programa prueba … End, el programa pertenece a L(G_ext). Las demás producciones y el símbolo inicial no cambian; G_ext = (V ∪ {<declaracion>, <tipo>}, Σ ∪ {entero, real}, P_ext, <programa>). La extensión define solo sintaxis: no comprueba declaraciones repetidas, declaración previa ni compatibilidad de

tipos. El archivo ejemplos/programa_extendido.txt (declaraciones, asignaciones y salidas) es aceptado por el validador con --extension, y la derivación desde <programa> consta en evidencias/derivaciones.txt.

4. Conclusiones

La GLC propuesta, con S = <programa>, representa programas con una o más instrucciones y distingue asignaciones simples, asignaciones con una operación aritmética y salidas. Se comprobó que ID y NUM, definidos como categorías léxicas, permiten reutilizar las mismas producciones para cualquier nombre o valor, y que las veinte instrucciones del programa de referencia quedan cubiertas. Un aprendizaje importante fue que la pertenencia depende del símbolo desde el que se comprueba: las instrucciones sueltas pertenecen a L(<instruccion>) pero no a L(<programa>), y por eso cada resultado indica contra qué lenguaje se verificó.

Las derivaciones por la izquierda y el árbol de derivación mostraron cómo cada paso aplica una producción y cómo las hojas reconstruyen la cadena. La validación de los ocho casos mostró que los errores se localizan en un elemento sintáctico concreto (falta de =, operador mal ubicado, ID faltante, # faltante). La extensión de declaraciones se incorporó formalmente con dos terminales, dos no terminales y una producción modificada, manteniendo la coherencia con el resto de la gramática. Queda fuera del alcance la semántica (tipos, variables declaradas y evaluación).

5. Consideraciones de entrega

- Informe en formato PDF (con copia editable en DOCX), con la numeración de secciones de la plantilla (1 a 5).
- Derivaciones, árboles y validaciones se generan con la misma gramática (gramatica.bnf) y el mismo validador.
- No se utilizó JFLAP; los árboles se generaron con Graphviz (.dot, .svg y .png en el repositorio).

Anexo – Evidencias

Repositorio GitHub: https://github.com/der-kaffe/info1148-laboratorio-3

Carpeta de Drive: https://drive.google.com/drive/folders/1SXU1cSjy1-m_Fg0xZlNUexpftVtnUft6?usp=sharing

Contenido: gramatica.bnf, validador.py, ejemplos/ (programa de referencia, programa extendido y casos), evidencias/ (validacion.json, derivaciones.txt, árboles en JSON y pruebas.txt con 25 pruebas automatizadas) y arbol_sintactico/ (.dot, .svg, .png).

A.1 Gramática léxica a nivel de carácter

Los terminales ID y NUM se definen también con producciones libres de contexto sobre caracteres, con ... como abreviatura de los rangos. ID es <id> con la restricción léxica de no coincidir con una palabra reservada, y NUM es <num>. Con estas reglas la sustitución léxica de las derivaciones es una derivación más; por ejemplo, <id> ⇒ <letra> <resto_id> ⇒ z <resto_id> ⇒ z.

```
<id>       → <letra> <resto_id>
<resto_id> → ε | <letra> <resto_id> | <digito> <resto_id> | _ <resto_id>
<num>      → <digitos> | <digitos> . <digitos>
<digitos>  → <digito> | <digito> <digitos>
<letra>    → A | B | ... | Z | a | b | ... | z | Ñ | ñ | Á | É | Í | Ó | Ú | Ü | á | é | í | ó | ú | ü
<digito>   → 0 | 1 | 2 | ... | 9
```

A.2 Resultado del validador en los casos del enunciado

Entrada	Respuesta del validador
Caso 1 `x = 10 #`	Válido: pertenece a L(<instruccion>).
Caso 2 `z = x + y #`	Válido: pertenece a L(<instruccion>).
Caso 3 `salida z #`	Válido: pertenece a L(<instruccion>).
Caso 4 `x 10 #`	Inválido para L(<instruccion>) (línea 1, columna 3): Se esperaba «=» después del identificador; se encontró «10».

Caso 5 `z = + x y #`	Inválido para L(<instruccion>) (línea 1, columna 5): Se esperaba un identificador o un número sin signo; se encontró «+».
Caso 6 `salida #`	Inválido para L(<instruccion>) (línea 1, columna 8): Se esperaba un identificador después de «salida»; se encontró «#».
Caso 7 `x = 20`	Inválido para L(<instruccion>) (línea 1, columna 7): Se esperaba «#» para terminar la asignación, o un operador aritmético; se encontró el fin de la entrada.
Caso 8, una línea por componente `Begin ⏎ Programa prueba ⏎ x = 5 # ⏎ salida x # ⏎ End`	Válido: pertenece a L(<programa>).
Caso 8, con / literales `Begin / Programa prueba / x = 5 # / salida x # / End`	Inválido para L(<programa>) (línea 1, columna 7): Se esperaba «Programa» después de «Begin»; se encontró «/».
Dos operadores `x = a + b * c #`	Inválido para L(<instruccion>) (línea 1, columna 11): Se esperaba «#» después de los dos operandos (solo se admite una operación); se encontró «*».
Signo unario `x = -1 #`	Inválido para L(<instruccion>) (línea 1, columna 5): Se esperaba un identificador o un número sin signo; se encontró «-».
Lista vacía `Begin Programa prueba End`	Inválido para L(<programa>) (línea 1, columna 23): Se esperaba al menos una instrucción; se encontró «End».
Dos instrucciones `x = 1 # y = 2 #`	Inválido para L(<instruccion>) (línea 1, columna 9): Se esperaba el fin de la entrada; se encontró «y».
Instrucción sola como programa `z = x + y #`	Inválido para L(<programa>) (línea 1, columna 1): Se esperaba «Begin» (distingue mayúsculas y minúsculas); se encontró «z».

A.3 Cómo repetir las comprobaciones

El comando python3 -m unittest discover -s pruebas ejecutó 25 pruebas con resultado OK (detalle en evidencias/pruebas.txt), y generar_evidencias.py reproduce las 19 comprobaciones de cadenas, los dos programas completos, las derivaciones y los árboles.

```
python3 validador.py ejemplos/programa_referencia.txt
python3 validador.py --fragmento --cadena "z = x + y #"
python3 validador.py ejemplos/programa_extendido.txt --extension
python3 herramientas/derivacion.py --fragmento --desde instruccion --cadena "z = x + y #"
python3 -m unittest discover -s pruebas -v
python3 herramientas/generar_evidencias.py
```

Programa de referencia validado:

```
Begin
Programa operativo_01_02
x = 1.345 #
y = 20 #
/* operaciones y salidas */
z = x + y #
z2 = x + y #
h = z - x #
h2 = z - x #
k = h * y #
t = k / z #
m = k % 10 #
salida z #
salida h #
salida k #
salida t #
salida m #
temporal = 0 #
temporal = x #
x = y #
y = temporal #
salida x #
salida y #
End
```